

The `.meshgrid()` method is used to calculate a two-variable function in Python. Now let's say we want to plot the following function consisting of x and y variables.

$$Z = \cos\left(\frac{\sqrt{X^2 + Y^2}}{\pi}\right)$$

Let X and Y values be between -10 and 10. First, we generate X and Y values with the `np.linspace()` method.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt

u = np.linspace(-10,10,100)
v = np.linspace(-10,10,100)
```

Now we will create (X,Y) pairs by creating the number sequences u and v. We will use the `np.meshgrid()` function for this. So we can use the X and Y variables in the above function.

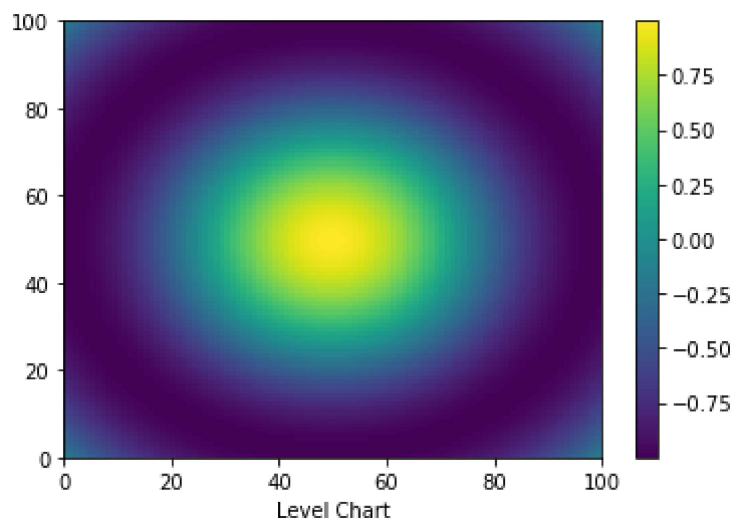
```
In [2]: X, Y = np.meshgrid(u,v)
```

Let's calculate the Z variable by using the X and Y variables in the above equation. Pay attention to the Numpy functions we use. Let's use the `np.hypot(X,Y)` function instead of the `np.sqrt(X2 + Y2)` function in the equation.

```
In [10]: Z = np.cos(np.hypot(X,Y)/np.pi)
```

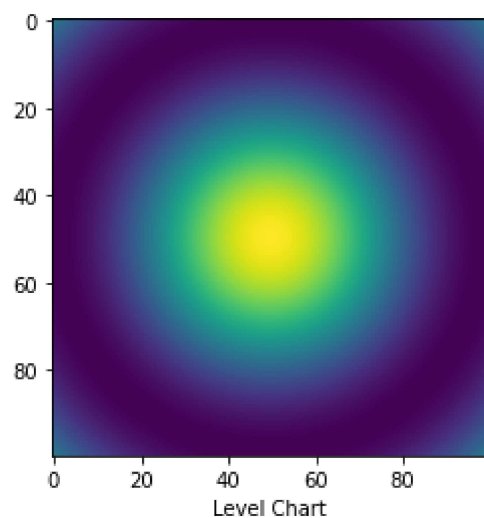
Now let's draw the level graph. We can use `plt.pcolor()` or `plt.imshow()` functions for this. Below are graphs drawn with both. Dark colors show low values in pixels and light colors show high values. You can do this with `plt.colorbar()` You can also tell from the color scale we created.

```
In [11]: plt.pcolor(Z)
plt.colorbar()
plt.xlabel('Level Chart')
plt.show()
```



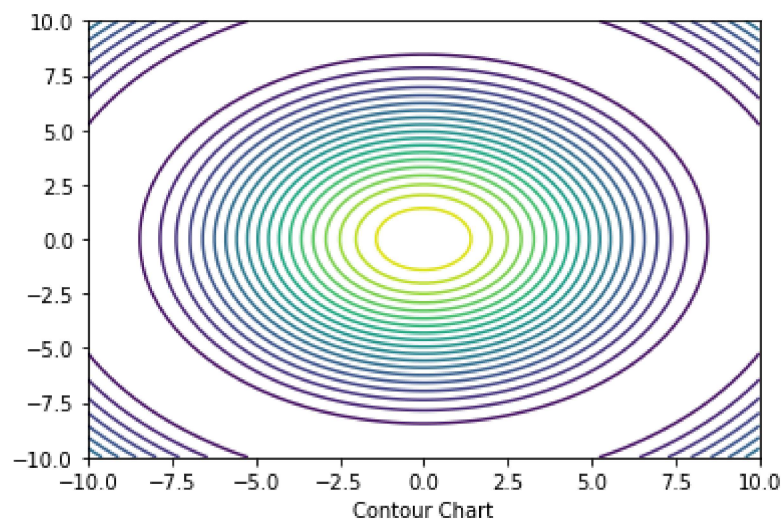
Now let's plot the graph with `plt.imshow()`.

```
In [12]: plt.imshow(Z)
plt.xlabel('Level Chart')
plt.show()
```



We see the values as pixels in the above drawings. We can also draw the same graph as level graphs. In the level graph below, Z has the same values on a curve. If we want, we can specify the number of curves to be drawn. For example, if we write `plt.contour(X,Y,Z,20)`, there are 20 curve is drawn.

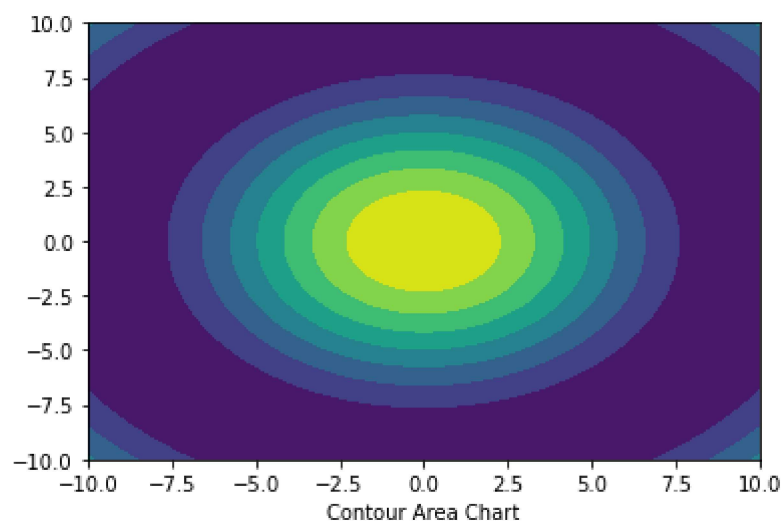
```
In [20]: plt.contour(X,Y,Z,20)
plt.xlabel('Contour Chart')
plt.show()
```



A mix of the above two chart types is the `plt.contourf()` function for plotting areas between levels.

In [19]:

```
plt.contourf(X,Y,Z)
plt.xlabel('Contour Area Chart')
plt.show()
```



In the line graph, the data points are combined with the line segments. Instead, the graphs that show each data point individually in shit or any other way we want are called point graphs or scatter plots. The economic growth data of Turkey between 1999-2016 in the previous section Let's draw it as a scatter plot. For the scatter plot, it will be enough to use the `.scatter()` method instead of the `.plot()` method.

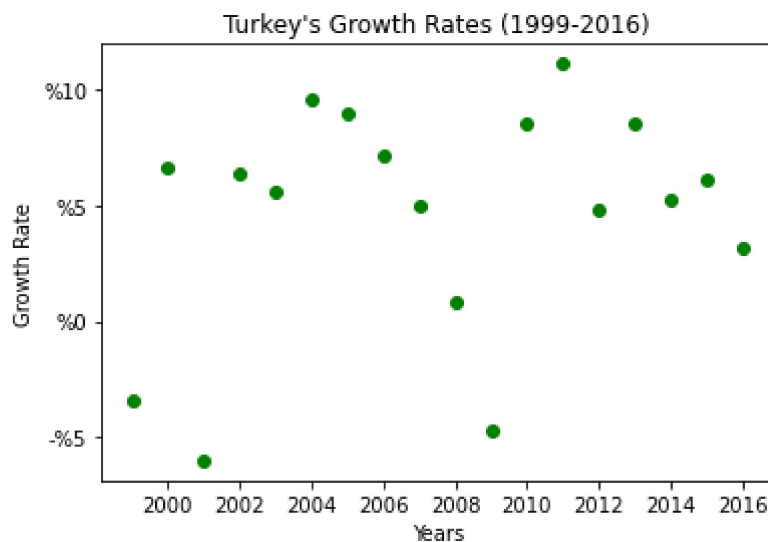
In [7]:

```
import matplotlib.pyplot as plt

years = range(1999,2017)

growth = [-3.4,6.6,-6,6.4,5.6,9.6,9,7.1,5,0.8,-4.7,8.5,11.1,4.8,8.5,5.2,6.1,3.2]

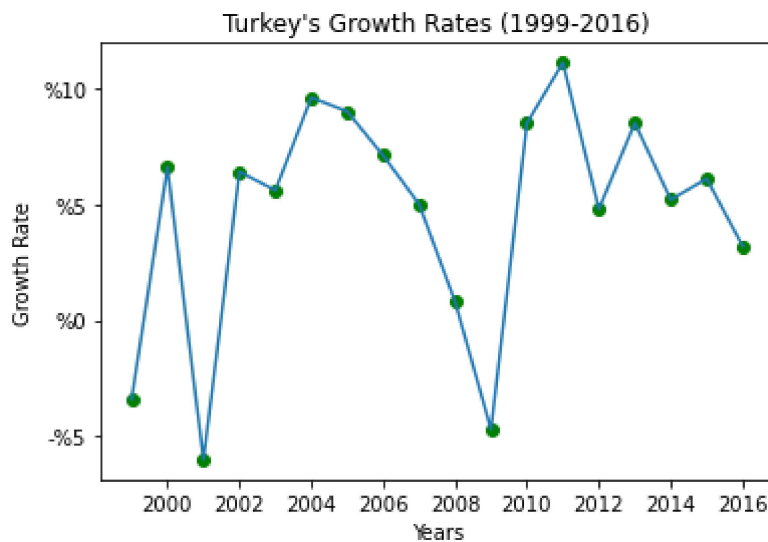
plt.scatter(years,growth,color='green')
plt.xticks([2000,2002,2004,2006,2008,2010,2012,2014,2016])
plt.yticks([-5.0,0.0,5.0,10.0],['-%5','%0','%5','%10'])
plt.xlabel("Years")
plt.ylabel("Growth Rate")
plt.title("Turkey's Growth Rates (1999-2016)")
plt.show()
```



We can use `.scatter()` and `.plot()` method together to connect points with lines.

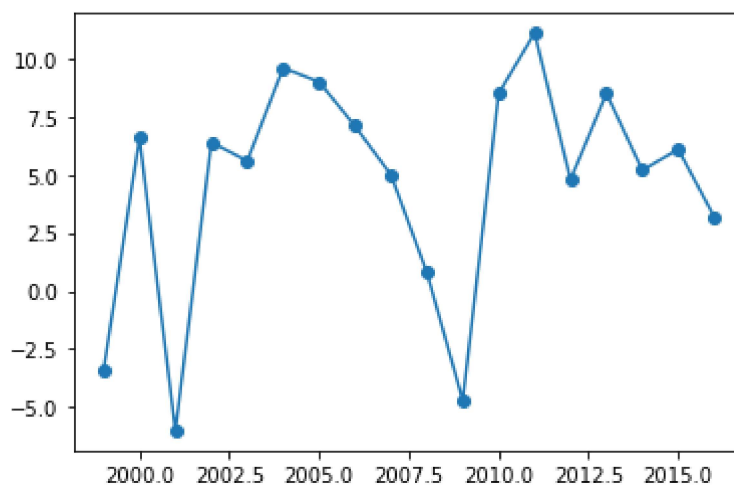
In [10]:

```
plt.scatter(years,growth,color='green')
plt.plot(years,growth)
plt.xticks([2000,2002,2004,2006,2008,2010,2012,2014,2016])
plt.yticks([-5.0,0.0,5.0,10.0],['-%5','%0','%5','%10'])
plt.xlabel("Years")
plt.ylabel("Growth Rate")
plt.title("Turkey's Growth Rates (1999-2016)")
plt.show()
```

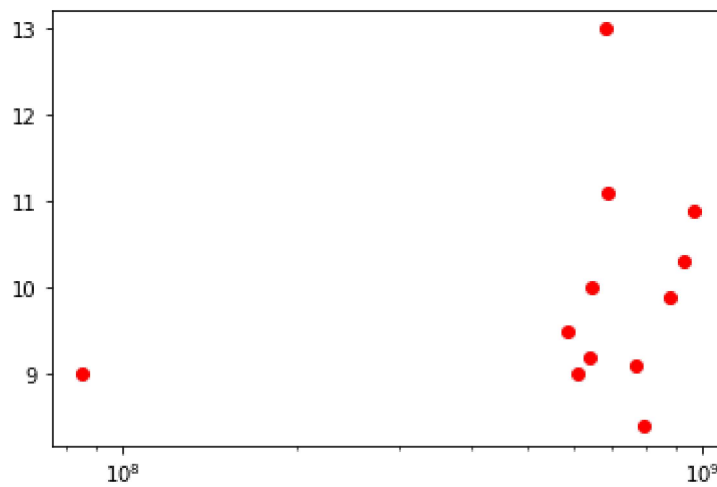
We can use point and line graphs together more practically. Let's look at the code below.

```
In [14]: plt.plot(years,growth,'-o')
plt.show()
```



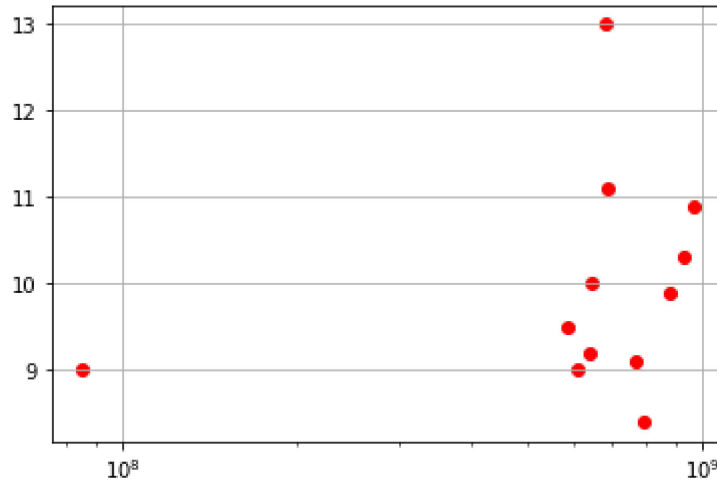
Sometimes we may want to compare two different variables, one on the x-axis and the other on the y-axis. In the example below, we compare the unemployment rate and income of Turkey between 2005-2016.

```
In [15]: unemployment =[9.5,9.0,9.2,10.0,13.0,11.1,
                        9.1,8.4,9.0,9.9,10.3,10.9]
income = [587895795,609464629,641608502,643416679,
          679462058,686235622,770301911,794629149,
          85238920,882760949,930677217,964776578]
plt.scatter(income,unemployment,color='red')
plt.xscale('log')
plt.show()
```



We can add level lines in addition to the chart. For this we add the `plt.grid(True)` line.

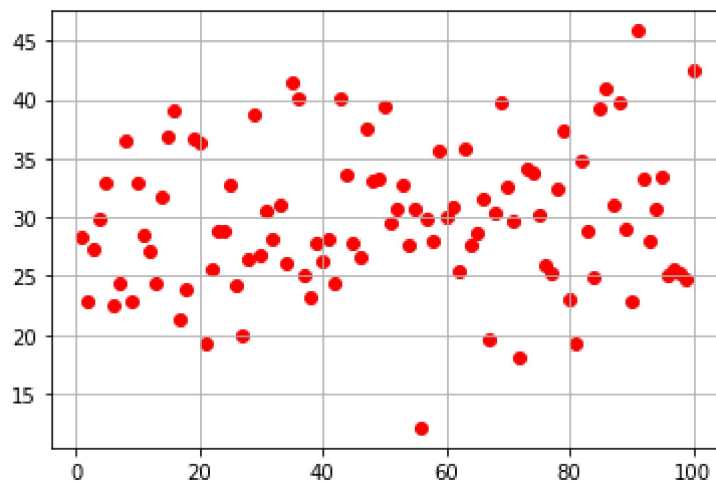
```
In [16]: unemployment = [9.5, 9.0, 9.2, 10.0, 13.0, 11.1,
                        9.1, 8.4, 9.0, 9.9, 10.3, 10.9]
income = [587895795, 609464629, 641608502, 643416679,
          679462058, 686235622, 770301911, 794629149,
          85238920, 882760949, 930677217, 964776578]
plt.scatter(income, unemployment, color='red')
plt.grid(True)
plt.xscale('log')
plt.show()
```



```
In [22]: import numpy as np

x = np.arange(1, 101)
y = 10 + 2 * np.random.normal(10, 3, 100)

plt.scatter(x, y, color='red')
plt.grid(True)
plt.show()
```



From time to time, we want to show two properties on the x and y axes in a scatterplot, while we want to show the third property as color or size. In the example we will use for this below, we will use the Boston house prices dataset from the sample datasets in the sklearn package. While showing the average price on the x-axis and the average price on the y-axis, we will reflect the crime rate in the city as size. We will use the sklearn.datasets package to get the dataset, the pandas package to make it available for plotting, and the matplotlib.pyplot packages for plotting.

```
In [38]: import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_boston

boston_housing = load_boston()
boston = pd.DataFrame(boston_housing.data)
boston.columns = boston_housing.feature_names
boston['Price'] = boston_housing.target

plt.scatter(boston.RM, boston.Price, s = boston.CRIM, color='red')
plt.xlabel('Average Number Of Rooms')
plt.ylabel('Price')
plt.show()
```

C:\ProgramData\Anaconda3\envs\batuhan\lib\site-packages\sklearn\utils\deprecation.py:87: FutureWarning: Function load_boston is deprecated; `load\_boston` is deprecated in 1.0 and will be removed in 1.2.

The Boston housing prices dataset has an ethical problem. You can refer to the documentation of this function for further details.

The scikit-learn maintainers therefore strongly discourage the use of this dataset unless the purpose of the code is to study and educate about ethical issues in data science and machine learning.

In this special case, you can fetch the dataset from the original source::

```
import pandas as pd
import numpy as np
```

```
data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
```

Alternative datasets include the California housing dataset (i.e. `:func:`~sklearn.datasets.fetch_california_housing``) and the Ames housing dataset. You can load the datasets as follows::

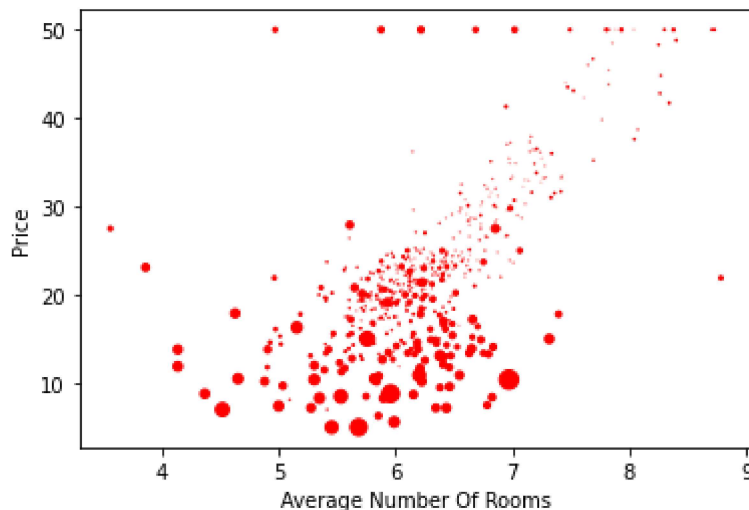
```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
```

for the California housing dataset and::

```
from sklearn.datasets import fetch_openml
housing = fetch_openml(name="house_prices", as_frame=True)
```

for the Ames housing dataset.

```
warnings.warn(msg, category=FutureWarning)
```



As seen in the graphic above, the crime rates in each city are reflected in the size of the figure. We do this with the `s`, that is, 'size' argument in the `plt.scatter()` function.

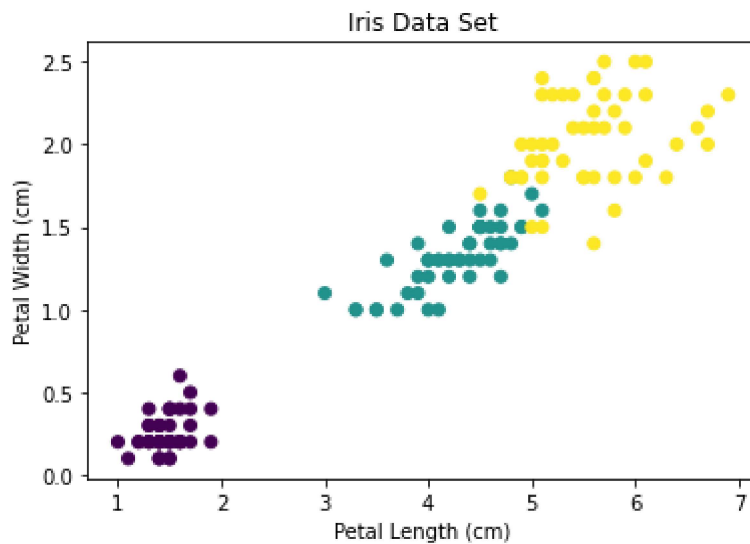
Now let's look at an example where we show different species with different colors. In the example below, the Petal Width and Petal Length features in the iris dataset, which is widely used in artificial learning, are drawn on the x and y axes, and different types of flowers are shown in different colors. A different color will be used for each different species in the column.

```
In [13]: import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import pandas as pd

iris_data = load_iris()
iris = pd.DataFrame(iris_data.data)
iris.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
iris['species'] = iris_data.target

plt.scatter(iris.petal_length, iris.petal_width, c=iris.species)
```

```
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Iris Data Set')
plt.show()
```



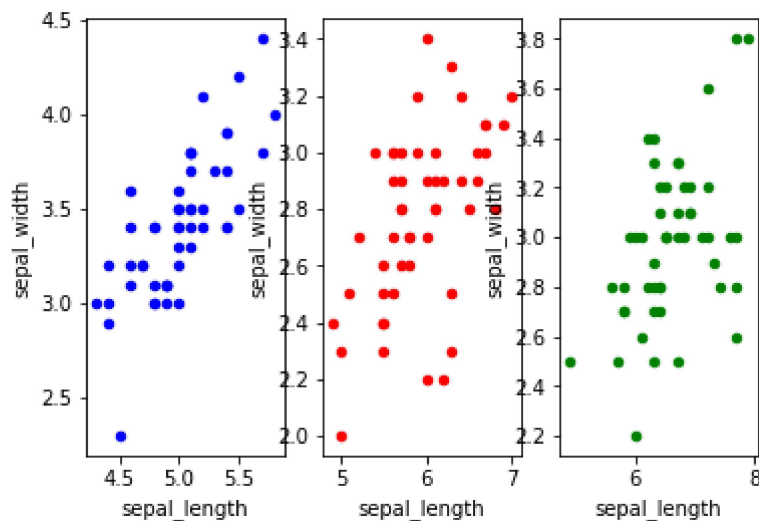
From the graph, it can be seen that there is a high and positive correlation between petal length and petal width. It is also seen that iris species in different groups are clustered in different areas. As a result, the type of this flower can be predicted with the width and height information.

Let's plot the groups in the iris data set in a slightly different way this time with the subplot method.

```
In [24]: #Let's split the chart into sub-charts consisting of 1 row and 3 columns.
figure,axis = plt.subplots(nrows=1,ncols=3)

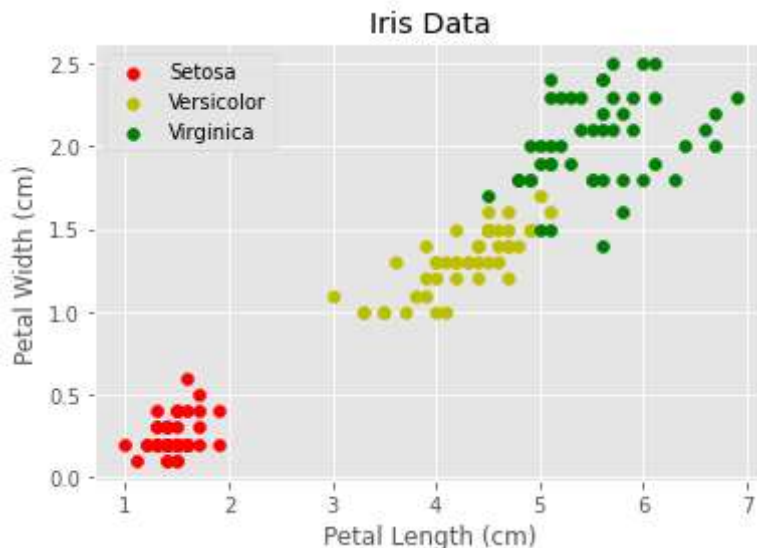
#Let's make a selection for each point and apply the .plot method.

iris[iris['species']=='Iris-setosa'].plot(ax = axis[0],x = 'sepal_length',y = 'sepal_
iris[iris['species']=='Iris-versicolor'].plot(ax = axis[1],x = 'sepal_length',y = 'se
iris[iris['species']=='Iris-virginica'].plot(ax = axis[2],x = 'sepal_length',y = 'sep
plt.show()
```



With the `style.use()` command in the matplotlib module, we can make the graphic plotted in different predefined themes. Now let's draw the iris graphic that we drew earlier in different themes (ggplot theme).

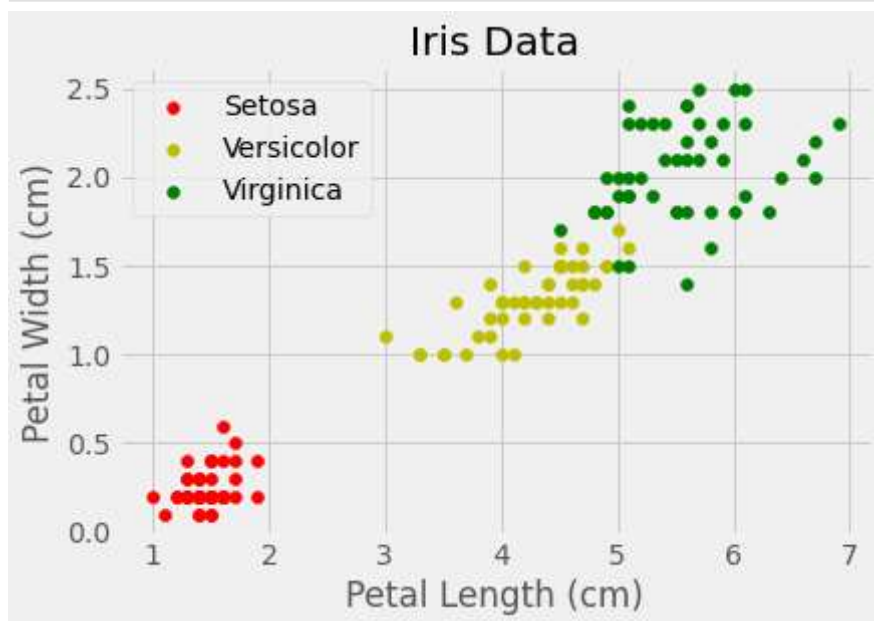
```
In [67]: plt.scatter(iris.iloc[0:49,2],iris.iloc[0:49,3],label='Setosa',c='r')
plt.scatter(iris.iloc[50:99,2],iris.iloc[50:99,3],label='Versicolor',c='y')
plt.scatter(iris.iloc[100:149,2],iris.iloc[100:149,3],label='Virginica',c='g')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Iris Data')
plt.legend(loc='upper left')
plt.style.use('ggplot')
plt.show()
```



The graphic above, drawn with another theme ('fivethirtyeight'), is as follows.

```
In [71]: plt.scatter(iris.iloc[0:49,2],iris.iloc[0:49,3],label='Setosa',c='r')
plt.scatter(iris.iloc[50:99,2],iris.iloc[50:99,3],label='Versicolor',c='y')
plt.scatter(iris.iloc[100:149,2],iris.iloc[100:149,3],label='Virginica',c='g')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
```

```
plt.title('Iris Data')  
plt.legend(loc='upper left')  
plt.style.use('fivethirtyeight')  
plt.show()
```



In []:

For the graphs we will draw, we will first need the pyplot subpackage in the matplotlib package. First we need to import this package. For convenience it is usually exported as plt for the matplotlib.pyplot package.

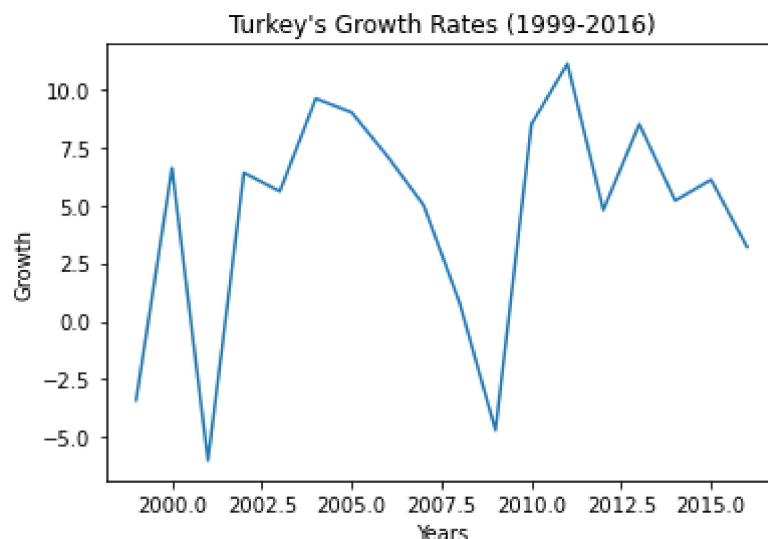
In the example below, let's draw a line graph of Turkey's economic growth rates between 1999 and 2016.

```
In [3]: import matplotlib.pyplot as plt
import numpy as np

years = np.arange(1999,2017)

growth = [-3.4,6.6,-6,6.4,5.6,9.6,9,7.1,5,0.8,-4.7,8.5,11.1,4.8,8.5,5.2,6.1,3.2]

plt.plot(years,growth)
plt.xlabel("Years")
plt.ylabel("Growth")
plt.title("Turkey's Growth Rates (1999-2016)")
plt.show()
```



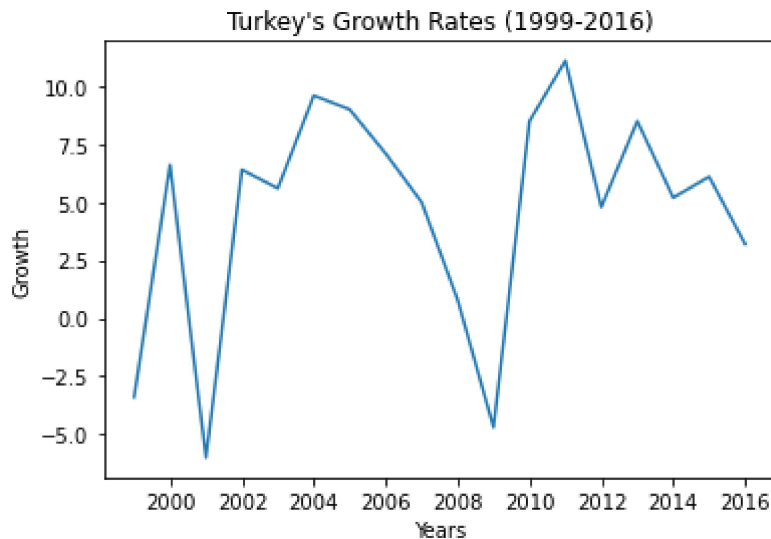
You may have noticed that the years are oddly written as 2002.5,2007.5 in the chart above. The .xticks() and .yticks() methods can be used to fix this.

```
In [2]: import matplotlib.pyplot as plt
import numpy as np

years = np.arange(1999,2017)

growth = [-3.4,6.6,-6,6.4,5.6,9.6,9,7.1,5,0.8,-4.7,8.5,11.1,4.8,8.5,5.2,6.1,3.2]

plt.plot(years,growth)
plt.xlabel("Years")
plt.ylabel("Growth")
plt.title("Turkey's Growth Rates (1999-2016)")
plt.xticks([2000,2002,2004,2006,2008,2010,2012,2014,2016])
plt.show()
```

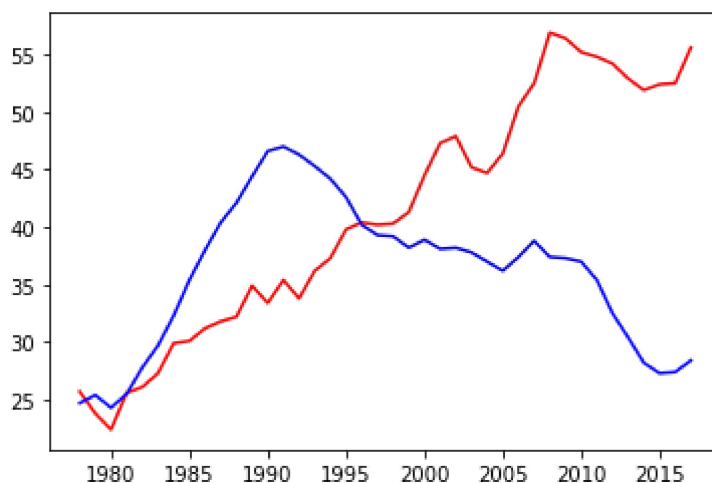



Now let's draw the two graphs together.

In [4]:

```
import numpy as np
import matplotlib.pyplot as plt

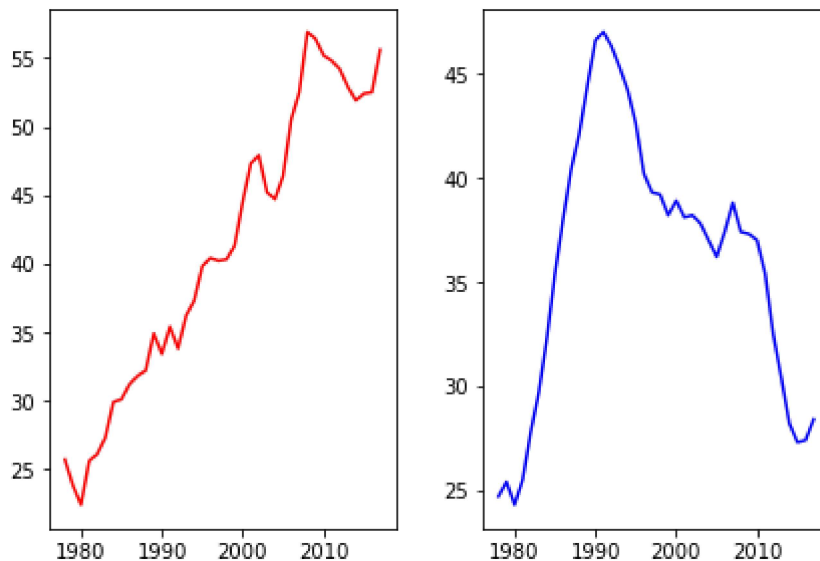
y1 = np.array([25.7,23.8,22.4,25.6,26.1,27.3,29.9,30.1,
               31.2,31.8,32.2,34.9,33.4,35.4,33.8,36.2,
               37.3,39.8,40.4,40.2,40.3,41.3,44.5,47.3,
               47.9,45.2,44.7,46.4,50.5,52.5,56.9,56.4,
               55.2,54.8,54.2,52.9,51.9,52.4,52.5,55.6])
y2 = np.array([24.7,25.4,24.3,25.5,27.8,29.7,32.3,35.4,
               38,40.4,42.1,44.4,46.6,47,46.3,45.3,
               44.2,42.6,40.2,39.3,39.2,38.2,38.9,38.1,
               38.2,37.8,37,36.2,37.4,38.8,37.4,37.3,
               37,35.4,32.5,30.4,28.2,27.3,27.4,28.4])
years = np.arange(1978,2018)
plt.plot(years,y1,'r') # r = red
plt.plot(years,y2,'b') # b = blue
plt.show()
```



It is possible to show the 2 graphics in the figure side by side in 2 different frames. We will use the `plt.axes()` method for this. In `plt.axes()` method, it is necessary to introduce 4 arguments in

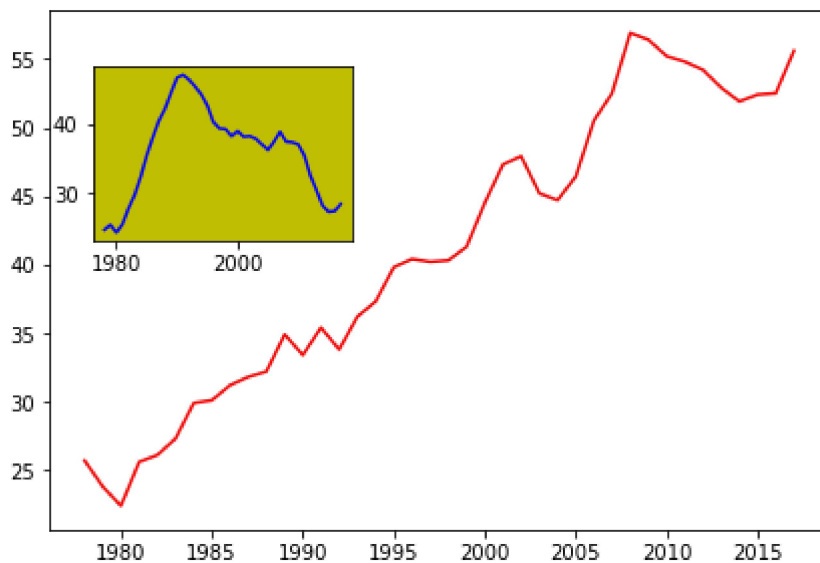
the list. `x,y,width,height`. The first 2 arguments, `x` and `y`, specify the position of the bottom left corner of the chart relative to the main chart. The width and height also represent the ratio of the width and height of the sub chart to the width and height of the main chart.

```
In [6]: plt.axes([0.05,0.05,0.4,0.9])
plt.plot(years,y1,'r')
plt.axes([0.55,0.05,0.4,0.9])
plt.plot(years,y2,'b')
plt.show()
```



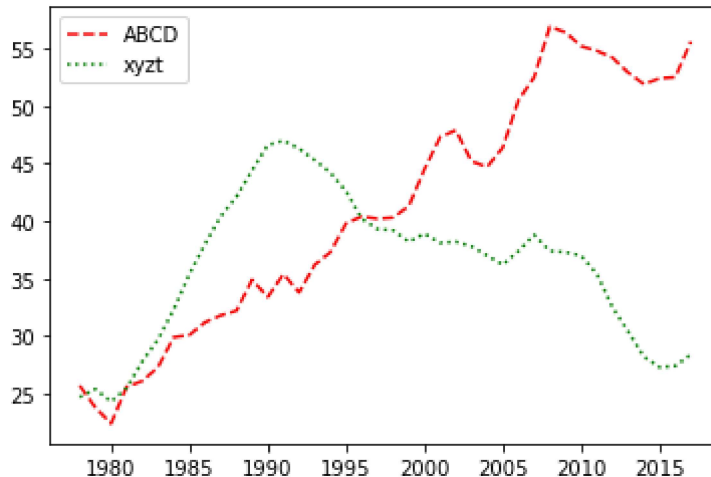
With the Axes method, it is also possible to draw another graphic in a smaller size inside a graphic. This allows creating beautiful visuals. Notice that we set the background color of the graphic with the `facecolor = 'y'` argument in the code below.

```
In [7]: plt.axes([0.05,0.05,0.9,0.9])
plt.plot(years,y1,'r')
plt.axes([0.1,0.55,0.3,0.3],facecolor = 'y')
plt.plot(years,y2,'b')
plt.show()
```



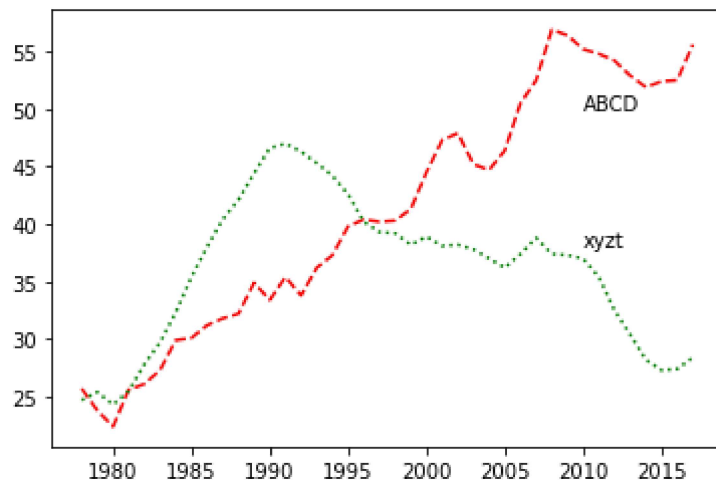
With the code below, we first specify the label we want to appear on the graph with the `label` argument for each data graph. We use the `plt.legend()` command to take place in the graph. To specify the location of the label, we set the `loc` argument as upper left.

```
In [9]: plt.plot(years,y1,'r--',label='ABCD')
plt.plot(years,y2,'g:',label='xyzt')
plt.legend(loc='upper left')
plt.show()
```



It is also possible to write text on the point we want in the graphics. This is a useful feature when we want to add annotations on the graphic. The `plt.annotate()` method is used to add text on the graphic. The text to be added in the method and the `xy` argument and the `x` and `y` coordinates where the text will be placed are transmitted. This time, instead of adding legends as above, let's write their names as text on the relevant graphics.

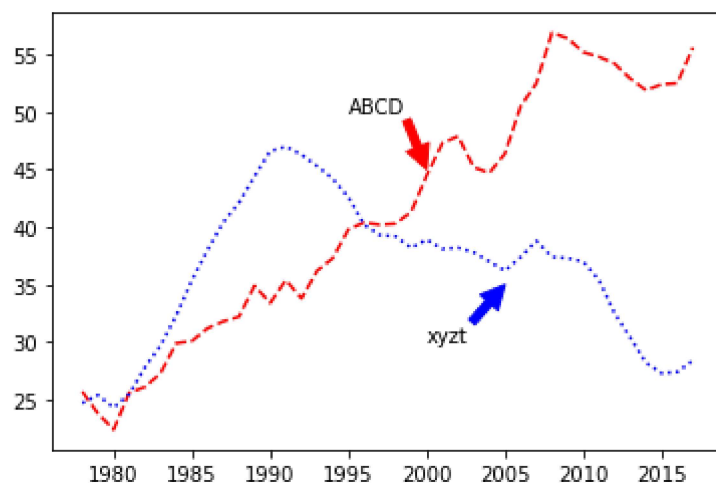
```
In [10]: plt.plot(years,y1,'r--')
plt.plot(years,y2,'g:')
plt.annotate('ABCD',xy = (2010,50))
plt.annotate('xyzt',xy=(2010,38))
plt.show()
```



It is also possible to draw arrows with the text. For this, the coordinates of the text with the `xytext` argument, the coordinates of the arrowhead with the `xy` argument, and the properties of the arrow with the `arrowprops` argument are transmitted as a dictionary. The `arrowprops` argument also makes the arrow appear. Therefore, we need to write this argument even if it is empty.

In [11]:

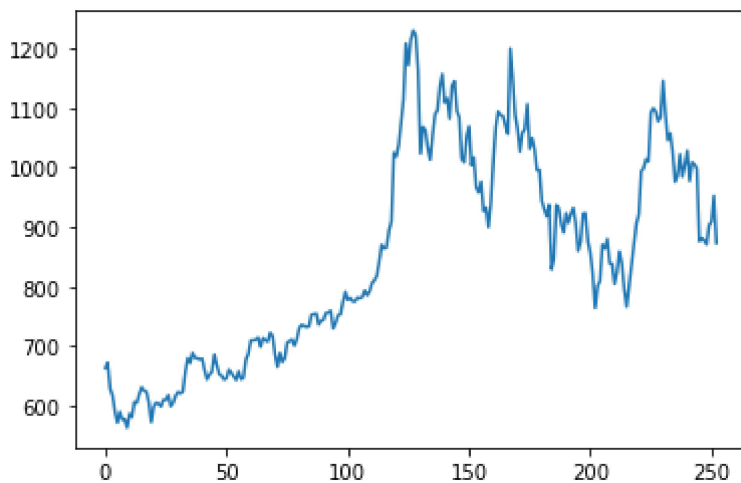
```
plt.plot(years,y1,'r--')
plt.plot(years,y2,'b:')
plt.annotate('ABCD',xytext=(1995,50),xy=(2000,45),arrowprops={'color':'red'})
plt.annotate('xyzt',xytext=(2000,30),xy=(2005,35),arrowprops = {'color':'blue'})
plt.show()
```



We can draw various graphs using the data of Tesla stock. For example, let's draw daily closing information.

In [89]:

```
import pandas as pd
import matplotlib.pyplot as plt
tesla = pd.read_csv('tesla.csv')
plt.plot(tesla['Close'])
plt.show()
```



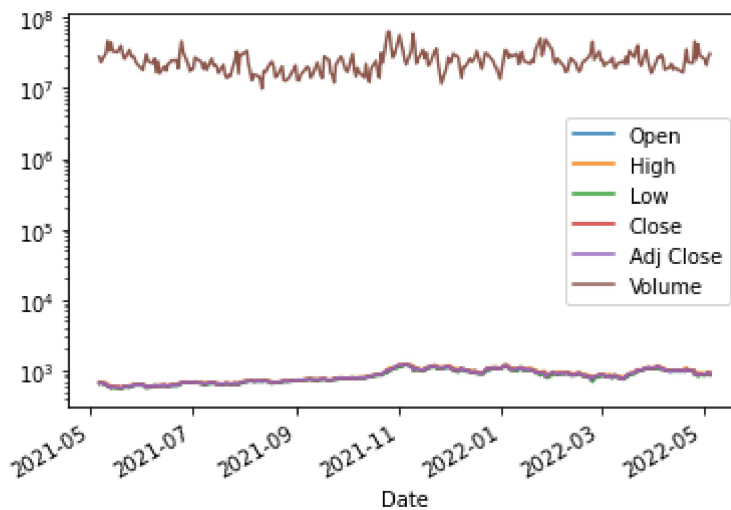
Now let's add the title and y-axis information to the chart.

```
In [53]: tesla = pd.read_csv('tesla.csv', index_col='Date', parse_dates = True)
tesla['Close'].plot(title = 'Tesla Closing Price')
plt.ylabel('Closing Price')
plt.xlabel('Date')
plt.show()
```



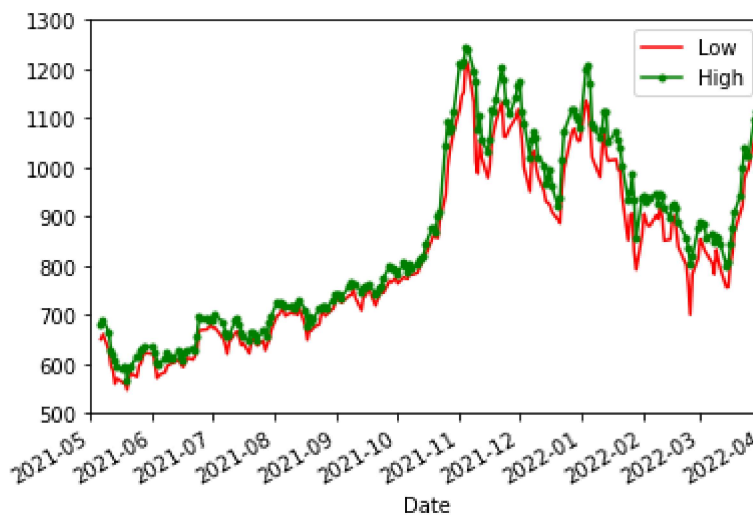
It consists of six different data such as time series, closing and volume of Tesla stock, if we want, we can plot them all at once.

```
In [88]: tesla = pd.read_csv('tesla.csv', index_col='Date', parse_dates = True)
tesla.plot()
plt.yscale('log')
plt.show()
```



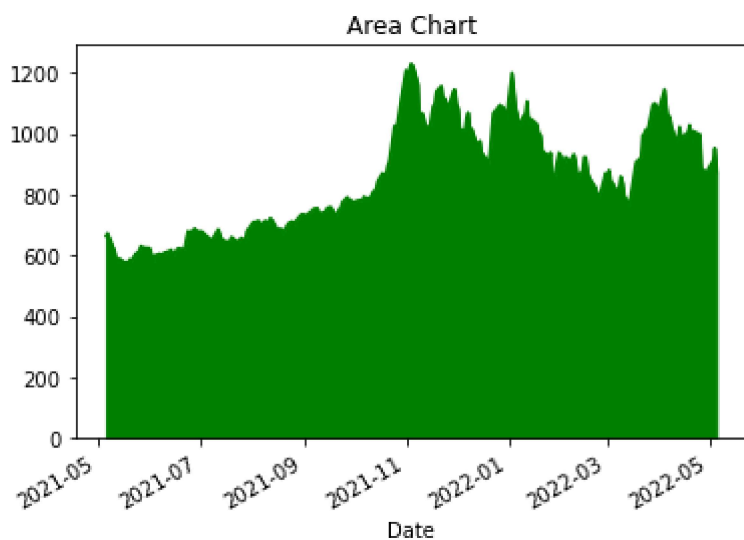
Now let's just plot the opening and closing graphs, we get data for 2015 and 2016.
`plt.axis([xmin,xmax,ymin,ymax])`

```
In [67]: tesla = pd.read_csv('tesla.csv', index_col='Date', parse_dates = True)
tesla['Low'].plot(color='red', style='-', legend=True)
tesla['High'].plot(color='green', style='-', legend=True)
plt.axis(('2021-05', '2022-04', 500, 1300))
plt.show()
```



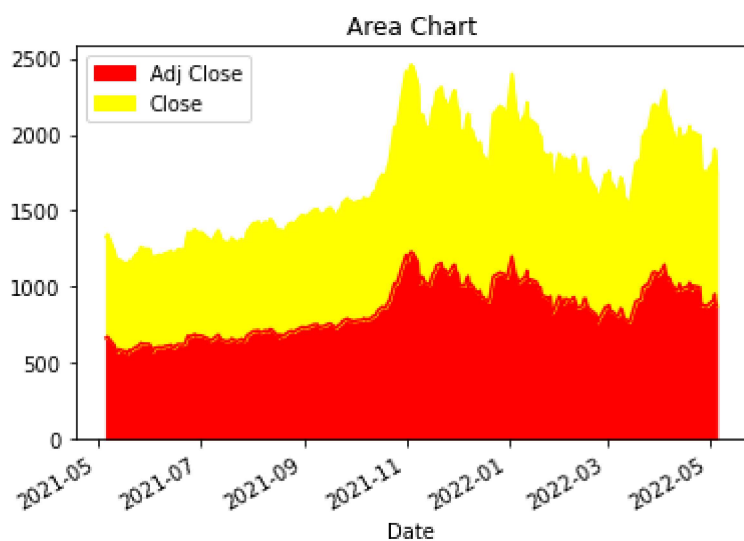
Let's draw an area chart, which is a different type of drawing.

```
In [74]: tesla = pd.read_csv('tesla.csv', index_col='Date', parse_dates = True)
tesla['Adj Close'].plot(kind='area', title='Area Chart', color='green')
plt.show()
```



Let's draw multiple columns together.

```
In [80]: tesla.loc[:,['Adj Close','Close']].plot(kind='area',title='Area Chart',color=['red',  
plt.show()
```



```
In [ ]:
```